Il modello Agile

Principi Fondamentali Agile

1. **Coinvolgimento del cliente (Customer Involvement)**
   - I clienti devono essere coinvolti durante tutto il processo di sviluppo.
   - Hanno il compito di fornire nuovi requisiti, prioritizzarli e **valutare le iterazioni** del sistema.
2. **Rilascio incrementale (Incremental Delivery)**
   - Il software è sviluppato in **versioni incrementali**.
   - Il cliente definisce quali requisiti devono essere soddisfatti in ciascuna versione.
3. **Persone prima dei processi (People, not Process)**
   - Le competenze del team di sviluppo devono essere **riconosciute e sfruttate**.
   - Il team è libero di sviluppare "a modo loro", senza vincoli eccessivi imposti da processi rigidi.
4. **Accettare il cambiamento (Embrace Change)**
   - I requisiti cambieranno inevitabilmente nel tempo.
   - Il sistema deve essere progettato per **favorire e adattarsi ai cambiamenti**.
5. **Mantenere la semplicità (Maintain Simplicity)**
   - Focalizzarsi sulla semplicità del software e del processo di sviluppo.
   - Eliminare la complessità non necessaria ogni volta che è possibile.

Interpretazione dei Principi Agile

1. Comunicazione

(correlata a Customer Involvement e People Not Process)

- Persone e interazioni sono più importanti di processi e strumenti.
- La comunicazione tra tutti gli attori di un progetto software è la **risorsa più importante**.
- Tutti possono comunicare con tutti: ad esempio, un **backend developer** può parlare direttamente con il cliente.
- Collaborare direttamente con i clienti porta a **migliori risultati rispetto ai soli rapporti contrattuali**.

---

2. Semplicità

(correlata a Maintain Simplicity)

- Mantenere la **descrizione formale del sistema** semplice e chiara.
- Avere **software funzionante** è più importante di documentazione estensiva.
- Il codice deve essere **semplice e tecnicamente avanzato**, riducendo la documentazione al minimo indispensabile.

---

3. Feedback

(correlata a Customer Involvement e Incremental Delivery)

- Rilasciare **nuove versioni del software** a intervalli frequenti.
- Testare il codice **sin dal primo giorno** per individuare problemi e correggerli rapidamente.

---

4. Coraggio

- Mettere il sistema in uso il prima possibile.
- Implementare **cambiamenti richiesti man mano**, invece di seguire rigidamente un piano iniziale.
- Rispondere ai cambiamenti è più efficace che pianificare tutto all'inizio e attenersi al piano.

Applicabilità dell’Agile

Quando conviene usare Agile

- Progetti di **piccola o media dimensione** (fino a circa 50 sviluppatori).
- Sviluppo di **sistemi custom** all’interno di un’organizzazione.
- Disponibilità dei **clienti a essere coinvolti** attivamente nel processo di sviluppo.
- Minore vincolo da **regole o regolamenti esterni** a cui il software deve aderire.

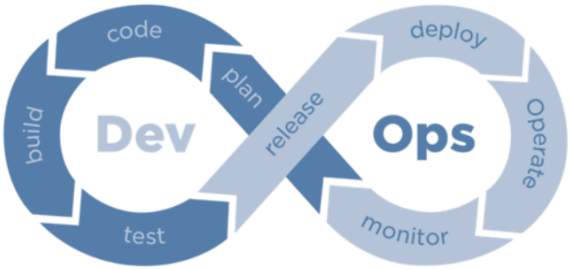
Quando viene realmente adottato

- Oggi, **quasi tutti i prodotti e le applicazioni moderne** sono sviluppati con approcci **agili**.

> Sintesi: Agile è particolarmente efficace quando il progetto richiede flessibilità, rapido rilascio e continuo coinvolgimento degli stakeholder, ed è diventato lo standard de facto per lo sviluppo software moderno.

Agile & CI/CD

L’adozione di metodologie **Agile** ha favorito la diffusione delle pratiche di **Continuous Integration (CI)** e **Continuous Deployment (CD)**.



Continuous Integration (CI) – Integrazione Continua

- Tutte le modifiche o aggiunte fatte dal team di sviluppo vengono **integrate continuamente** nel **main branch** del progetto.
- L'integrazione è **validata automaticamente** tramite strumenti di **automated build e test**.
- CI consente di **produrre continuamente nuove versioni** del software in sviluppo.

Continuous Deployment (CD) – Rilascio Continuo

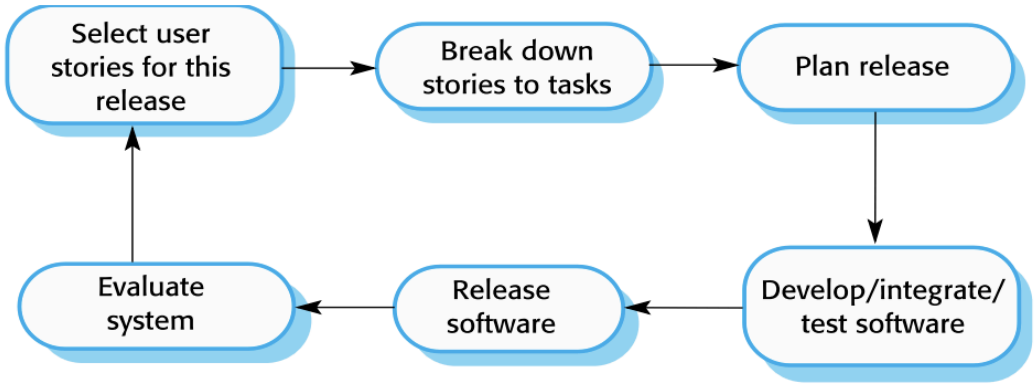
- Le versioni generate tramite CI vengono **automaticamente dispiegate** su ambienti di test o produzione.
- Garantisce un **rilascio rapido e frequente** delle nuove funzionalità.

Essere Agile: Extreme Programming (XP)

Extreme Programming (XP) è un approccio "estremo" allo sviluppo **iterativo** e **agile** del software. È stato progettato per massimizzare la qualità del software e la capacità di adattamento ai cambiamenti.

Caratteristiche principali di XP

- Nuove versioni di un sistema software possono essere sviluppate **più volte in un solo giorno**.
- Le versioni incrementali vengono rilasciate al cliente **ogni due settimane**, consentendo feedback continuo.
- **Tutti i test** sono eseguiti per ogni build; una build è accettata **solo se tutti i test passano**.
- Adotta un ciclo di rilascio iterativo molto rapido e orientato al feedback.



XP: Pratiche Comuni

1. Planning Incrementale

- Collezione dei requisiti sotto forma di **user stories**.
- Le user stories da includere in una release sono selezionate in base al **tempo disponibile** e alla **priorità**.
- Gli sviluppatori dividono le user stories in **task più piccoli**, facilmente gestibili.
- Questo supporta **Incremental Delivery** e consente di adattarsi ai cambiamenti.

2. Release Piccole

- Prima versione minima che fornisca **le funzionalità di base** del sistema.
- Rilascio **frequente e incrementale** di nuove funzionalità.
- Supporta sia l'**Incremental Delivery** sia la capacità di **Embrace Change**.

3. Design Semplice

- Progettazione mirata a soddisfare **solo i requisiti correnti**.
- Deve essere **comprensibile a tutti** i membri del team.
- Riferimento al principio **Maintain Simplicity**.

4. Test-First Development

- I casi di test vengono creati **prima del codice**.
- Spesso vengono generati tramite framework di **unit test automatici**.

- Supporta **Incremental Delivery** e la capacità di **Embrace Change**, perché ogni cambiamento viene immediatamente testato.

5. Refactoring

- Refactoring continuo appena si individuano possibili miglioramenti del codice.
- Obiettivo: codice **semplice, facilmente manutenibile e auto-esplicativo**.
- Motto: *"Se un metodo richiede un commento, riscrivilo!"*
- Supporta i principi **People Not Process** e **Maintain Simplicity**.

6. Pair Programming

- Gli sviluppatori lavorano **in coppia**.
- Garantisce **verifica continua e supporto reciproco** durante lo sviluppo.
- Riferimento a **People Not Process**.

7. Collective Ownership

- Tutte le coppie di sviluppatori lavorano su **tutte le aree del sistema software**.
- Tutti hanno **responsabilità su tutto il codice**: chiunque può modificare qualsiasi parte del sistema.
- Promuove collaborazione e condivisione del codice, in linea con **People Not Process**.

8. Sustainable Pace

- No al **lavoro straordinario eccessivo**.
- Presuppone che troppo lavoro straordinario porti a **calo della qualità del codice e della produttività**.
- Supporta **People Not Process** e **Maintain Simplicity**.

9. On-Site Customer

- Un rappresentante del cliente deve essere **disponibile o parte del team XP**.
- Responsabile di fornire e prioritizzare **nuovi requisiti** del sistema.

- Supporta il principio di **Customer Involvement**.

Diffusione e Utilizzo di XP

- XP si concentra fortemente su aspetti **tecnici** dello sviluppo software.
- A causa della sua intensità tecnica, **non è facilmente integrabile** con le pratiche di gestione utilizzate nella maggior parte delle organizzazioni.
- Sebbene i concetti XP siano **influenzati negli approcci agili moderni**, il metodo originale non è molto diffuso o utilizzato integralmente.

Pratiche chiave XP utilizzate comunemente

- **User stories** per la specifica dei requisiti.
- **Refactoring** continuo.
- **Test-first development**.
- **Pair programming**.
- **On-site customer**, seppur in forma limitata con interazioni periodiche.

Essere Agile: Scrum

Scrum è un metodo agile per lo sviluppo **iterativo e incrementale** di sistemi software.

- Il nome “Scrum” deriva dal termine usato nel rugby per indicare la **mischia**, simbolo di un gruppo che lavora all’unisono per avanzare insieme.
- L’obiettivo è ottenere un processo in cui il team si muove **in sincronia** per raggiungere un risultato che soddisfi sia:
  - **Le ambizioni del team**, sia
  - **Le ambizioni individuali dei membri**.

Scrum: Tre figure coinvolte

1. Product Owner

- Individuo (o piccolo gruppo) responsabile di identificare le **caratteristiche del prodotto software** da sviluppare.
- Compiti principali:
  - Elencare e **prioritizzare i requisiti**.
  - Revisionare il **product backlog** (insieme dei requisiti conosciuti) per garantire che il progetto rimanga allineato alle esigenze di business.

2. Scrum Master

- Figura responsabile di assicurare che il processo Scrum sia **seguito correttamente**.
- Compiti principali:
  - Garantire **condizioni ambientali e motivazioni** necessarie per il lavoro del team.
  - Interfacciarsi con il resto dell'organizzazione per ridurre interferenze esterne.
- Non ha autorità sul team, **non è un project manager**.

3. Development Team

- Gruppo **autogestito** di sviluppatori software.
- Non dovrebbe superare le **7 unità**, secondo il principio del **two-pizzas team** (un team che può essere sfamato con due pizze).
- Responsabile dello sviluppo del software e di eventuali documenti essenziali.

Scrum: Fasi

1. Pre-Game Phase (Pianificazione "di massima")

- **Planning sub-phase:**
  - Definizione del sistema da sviluppare tramite il **product backlog**, che contiene tutti i requisiti attualmente conosciuti.
- **Architecture sub-phase:**
  - Design di **alto livello del sistema**, inclusa l'architettura, basato sugli elementi del product backlog.

2. Game Phase (Sviluppo)

- Il sistema viene sviluppato attraverso una serie di **sprint**, cicli iterativi nei quali vengono implementate o migliorate funzionalità.
- Uno sprint dura da **una settimana a un mese**.
- Ogni sprint include le tradizionali **fasi di sviluppo del software** (specifica, progettazione, sviluppo, test).
- L'**architettura del sistema evolve** durante gli sprint, adattandosi ai cambiamenti e alle nuove funzionalità implementate.

3. Post-Game Phase (Chiusura della release)

- Conclude il processo di sviluppo e prepara il prodotto per il **rilascio finale**.
- Include attività come:
  - **Integrazione del sistema**
  - **Test finale**
  - **Documentazione per l'utente**
  - **Formazione**
  - Preparazione del **materiale di marketing**

Sprint in Scrum

Gli **sprint** sono cicli di sviluppo con **durata prefissata**, tipicamente **2-4 settimane**. Ogni sprint segue un flusso preciso che coinvolge team e clienti.

1. Partenza dallo **Sprint Backlog**

- Si parte dal **product backlog**, contenente la lista dei requisiti TBD (*To Be Done*), cioè ciò che deve ancora essere aggiunto al progetto.
- La **fase di selezione** coinvolge tutto il team e il cliente:
  - Si decide quali requisiti sviluppare nello sprint corrente.
- Output: **sprint backlog**, cioè i requisiti selezionati per lo sprint.

2. Pianificazione dello sprint

- Gestita dal **Product Owner**.
- Creazione dello **sprint backlog** dettagliato, con le attività da svolgere durante lo sprint.
- Da questo momento, il team lavora in **isolamento**, protetto dallo **Scrum Master** da distrazioni esterne, incluse comunicazioni con il cliente o altre aree dell'organizzazione.

3. Esecuzione dello sprint

- Lo **Scrum Master** organizza **brevi meeting giornalieri (daily scrum)**:
  - Cosa ho fatto ieri?
  - Cosa farò oggi?
  - Quali problemi sto affrontando?
- Il resto del tempo è dedicato allo **sviluppo effettivo del software**.

4. Revisione dello sprint

- Alla fine dello sprint, l'**incremento prodotto** viene revisionato:
  - Incontro di circa **4 ore**, con team e clienti/utenti.
  - Feedback su:
    - È questo il prodotto che vogliamo costruire?
    - Come reagirebbero gli utenti finali?
    - Il prodotto risponde ancora ai requisiti iniziali?

        - Ci sono cambiamenti o nuove idee da introdurre?
- Possibilità di **iniziare subito il prossimo sprint** con le modifiche o le priorità aggiornate.

5. Retrospettiva dello sprint

- Tra uno sprint e il successivo, si organizza un **evento di retrospettiva** (~3 ore).
- Obiettivo:
    - Riflettere sulle attività svolte.
    - Identificare problemi e soluzioni.
    - Migliorare continuamente il **processo e le competenze del team**.

